Research and Adaptation of Components from the Persistent OS Phantom for Genode

student: Mikhail Voronin
supervisor: Artem Burmyakov
consultant: Alexander Tormasov

June 2026

Relevance

- Orthogonal persistence has been developing since the 1980s
- New equipment spurs new research
- The current situation requires a research

Objective

Current implementation:

- Test systems exist
- Embedded systems
- No permanent physical access is required

Goal: Explore options for ensuring data security

Data Persistence

Persistence the period of time during which an object is usable[1]

Today's dominant model — **two-level store** (non-persistent):

- Data is either *active* (RAM) or *stored* (disk)
- Programmer manually moves data between levels

Consequences:

- Data loss
- I/O boilerplate (≈30% of code)
- Frequent initialization and configuration

[1] M. P. Atkinson, P. J. Bailey, K. J. Chisholm, W. P. Cockshott, and R. Morrison, "Ps-algol: A language for persistent programming," 1983.

Orthogonal Persistence

Alternative: the **environment** manages persistence

Principles of **orthogonal persistence**[2]:

1. **Independence**
2. **Data type orthogonality**
3. **Identification**

Result: No two-level store model's drawbacks and improved disk's bandwidth utilization

[2] R. Morrison and M. P. Atkinson, "Persistent Languages and Architectures," 1990.

PhantomOS

PhantomOS[3] is a non-Unix like OS implementing orthogonal persistence.[4]

- Phantom Virtual Machine (PVM): bytecode VM of a persistent language
- Simplified POSIX layer (non-persistent)
- Periodical snapshots (via **Copy-on-Write**)
- Changed pages written to disk (incremental)
- Programs are temporarily suspended (snapshot preparation)

[3] `http://phantomos.org/`

[4] D. Zavalishin, PhantomOS, 2010.

Genode

PhantomOS ported to the **Genode** OS C++ framework[5]:

- Proven microkernels (NOVA, seL4, Fiasco.OC)
- Capability-based security
- Minimal TCB

[5] `https://genode.org/`

Snapper

Snappser[6] is a file-system-based transactional snapshot mechanism[7]

Previous approach ("superblock"): snapshots stored as monolithic objects.

+ Fast
– No fault tolerance, no storage recovery

Snapper 2.0:

- Configured generations number
- Unchanged pages **linked**, not copied across generations
- ext4 + fsck for crash recovery
- Integrity checks (hashes per file)
- Configurable redundancy policy

[6] `https://github.com/rumenmitov/snapper`

[7] R. Mitov, "Snapper (v2): A PhantomOS snapshot mechanism," FOSDEM 2026. `https://fosdem.org/2026/schedule/event/DJ8YVT-practical-persistence/`

Problem Statement

Problem:

- Snapshots contain **sensitive data**
- **Plaintext** snapshots are stored on the drive

Consequences:

- Data can be **read directly**
- Ability to **restore the complete state**

Immediate decision's challenges:

- **No prior research** on this for PhantomOS
- **No ready-made solution** exists in Genode

Solution: to implement a mechanism for the secure storage of snapshots.

Environment & Requirements

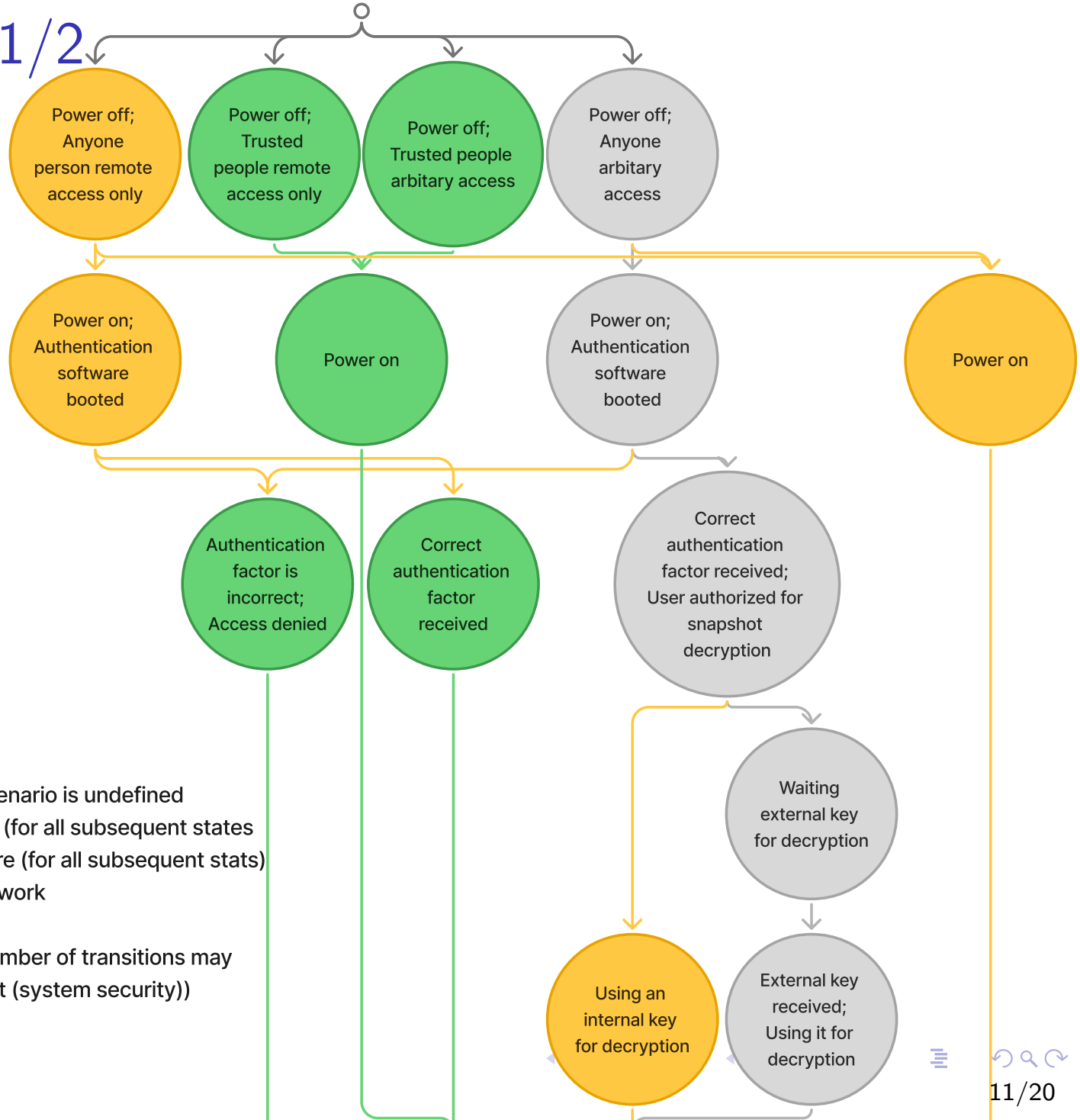
No unauthorized access to data

Requirements:

1. Mandatory authorization
2. Key stored on an **external USB drive**
3. Encryption of snapshots (AES, block-level)
4. Only encrypted data written to disk
5. Acceptable performance overhead ($\leq 2\times$)

Requirements 1 & 2 combined: the USB drive contains the key and serves as the authentication factor.

Usage scenarios 1/2



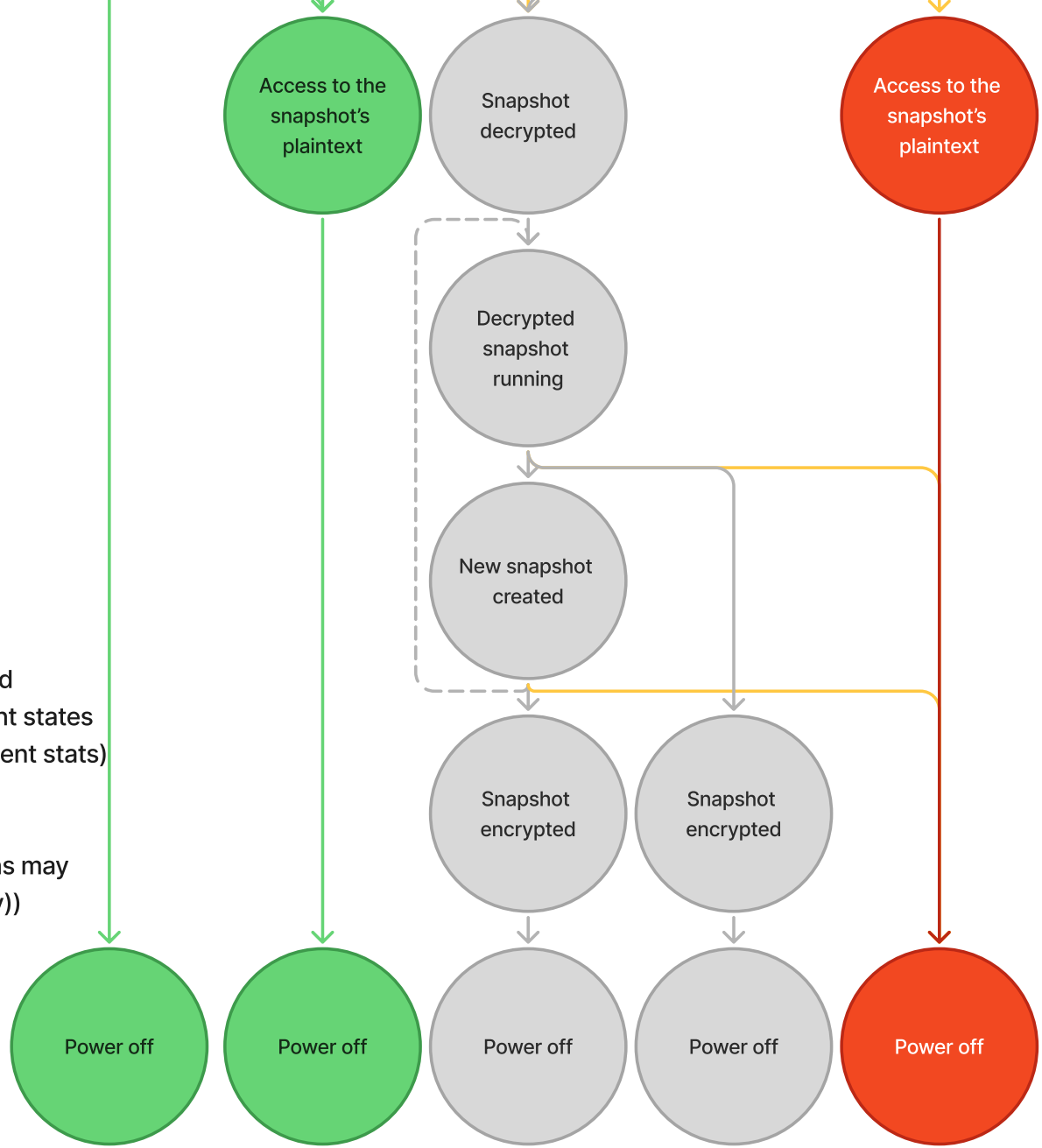
Legend:

- System state (computer - user)
- Note to the state
- In this state, the security of the scenario is undefined
- In this state the scenario is secure (for all subsequent states
- In this state the scenario is insecure (for all subsequent stats)
- This scenario is considered in our work
- Transition between states
- Transition between states (The number of transitions may vary and does not affect the final result (system security))

Usage scenarios 2/2

Legend:

- System state (computer - user)
- Note to the state
- In this state, the security of the scenario is undefined
- In this state the scenario is secure (for all subsequent states
- In this state the scenario is insecure (for all subsequent stats)
- This scenario is considered in our work
- Transition between states
- Transition between states (The number of transitions may vary and does not affect the final result (system security))



Implementation: se_vault

file_vault based on **Tresor** library:

- Tresor: AES encryption on 4K blocks, SHA-256 integrity, CoW atomicity
- file_vault: state orchestrator providing a secure FS session

Implemented **se_vault**[8] based on **file_vault**[9]:

- Headless operation
- Passphrase replaced by **USB key file**
- rump_fs/ext2 replaced by **lwext4/ext4**
- Added **fsck** crash recovery support with fsck
- Direct block session (skip image file creation)
- All block sizes standardized to 4K
- Graceful shutdown sequence added

[8] `https://github.com/VoronM1522/genode/tree/se/repos/gems/src/app/se_vault`

[9] `https://github.com/genodelabs/genode/tree/sculpt-25.04/repos/gems/src/app/file_vault`

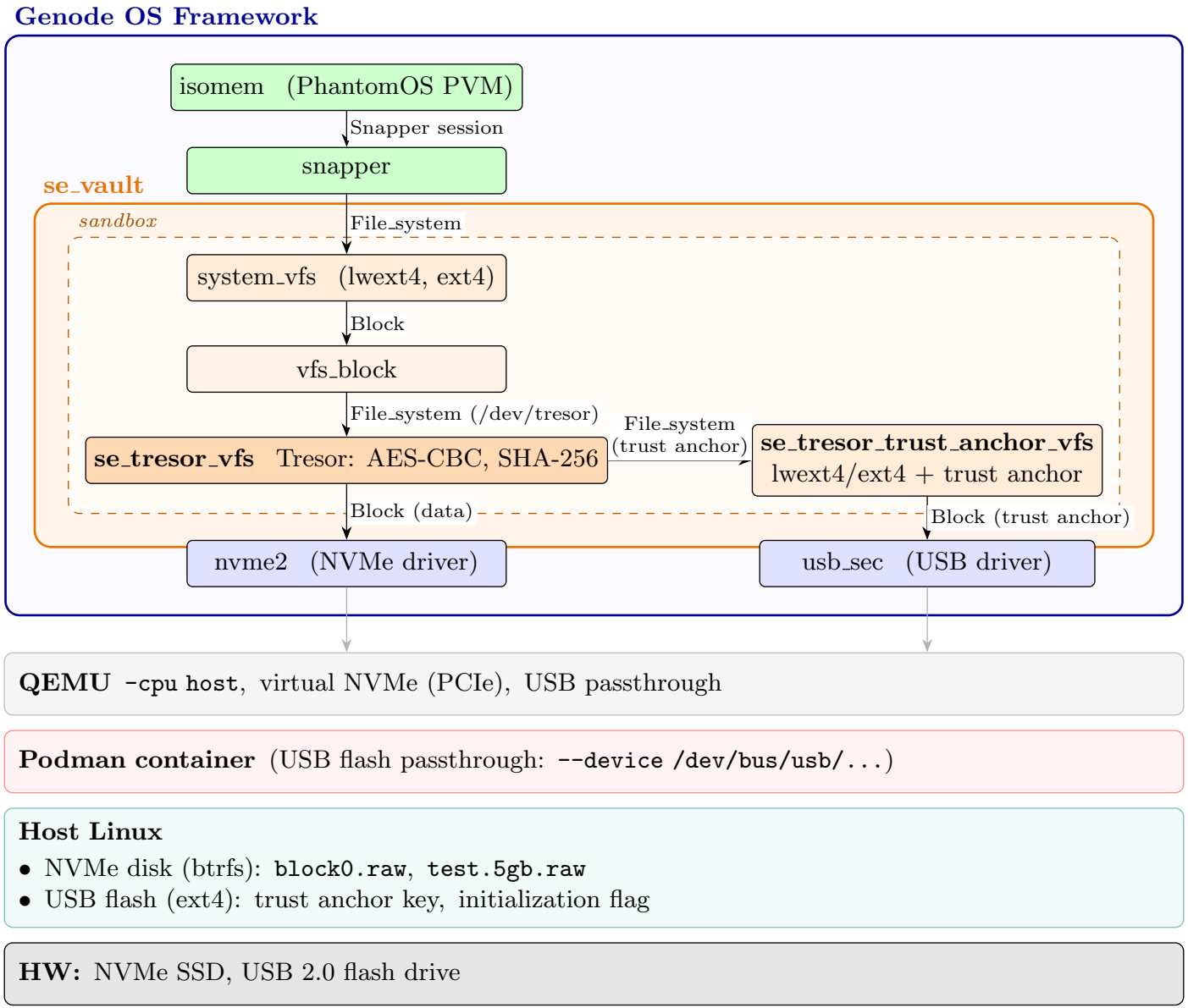
Related Fixes & Adaptations

Functionality patches:

- lwext4: `ftruncate` and `sync`
- isomem: power-off button callback and graceful shutdown
- snapper: client counter and graceful shutdown
- vfs: client counter, graceful shutdown and clean unmount

Optimization: using a "Block" session directly
Reduced initialization time from ≈20 min to ≈1 min

System Architecture



First run configuration

Environment features:

- QEMU with `-cpu host` (AES/SHA hardware acceleration)
- NVMe controllers
- 2 GB image for isomem
- 5 GB image for se_vault (4 GB user FS)

Functional testing

Functional tests (all passed):

- Normal boot / shutdown / restore cycle
- Encrypted image cannot be mounted without key
- Key/hash replacement attack — superblock not detected
- Crash recovery via `fsck`

Performance testing

Methodology: first snapshot creation and reboot with restoring, 5 runs each

Operation	Without enc.	With enc.	Overhead
Snapshot creation	2.10 s	51.00 s	$\approx 24\times$
Snapshot restore	2.50 s	27.30 s	$\approx 11\times$

The results are not statistically confirmed

Root cause: **known Tresor limitation** — no multithreading optimization

Evaluation

4 of 5 requirements met

- ✓ Mandatory authorization (USB key)
- ✓ Key on external medium
- ✓ AES block-level encryption
- ✓ Only encrypted data on disk
- × Overhead $\leq 2\times$ *(Tresor limitation)*

The current overheads may be acceptable in certain scenarios

Conclusion

Developed **se_vault** based on **file_vault**
Improvements:

- Security gain: snapshot encryption enabled
- Integrity gain: SHA-256 at block level (Tresor) + hash at FS level (Snapper)

First security research specifically for PhantomOS

Future work:

- Decrease performance overhead
- Encrypt the isomem's block device
- Investigate merging Snapper and Tresor snapshot concepts
- Bring the PoC implementation to an industrial-scale level